

Assessment Programs

Questions 1 to 5 — Python Source Code

Question 1: Water Jug Problem — Goal-Based Agent

```
class WaterJugAgent:
    def __init__(self, jug4_capacity, jug3_capacity):
        self.jug4_capacity = jug4_capacity
        self.jug3_capacity = jug3_capacity
        self.jug4 = 0
        self.jug3 = 0
    def solve(self):
        self.jug3 = self.jug3_capacity
        self.jug4 = self.jug3
        self.jug3 = 0
        self.jug3 = self.jug3_capacity
        transfer = min(self.jug4_capacity - self.jug4, self.jug3)
        self.jug4 += transfer
        self.jug3 -= transfer
        self.jug4 = 0
        self.jug4 = self.jug3
        self.jug3 = 0
    def display(self):
        print("4-Gallon Jug:", self.jug4)
        print("3-Gallon Jug:", self.jug3)
        print("Goal Achieved: 2 Gallons in 4-Gallon Jug")
        print("Agent Type: Goal-Based AI Agent")
```

Question 2: Smart Farm Agent — Model-Based Goal-Oriented Agent

```
class SmartFarmAgent:
    def __init__(self):
        self.percepts = []
    def perceive(self, percept):
        self.percepts.append(percept)
    def display_percepts(self):
        for p in self.percepts:
            print("Percept:", p)
    def goal(self):
        print("Goal: Monitor Farm Conditions")
    def agent_type(self):
        print("Agent Type: Model-Based Goal-Oriented Agent")
```

Question 3: 8-Queens Problem — Backtracking Search

```
N = 8
board = [ "." * N for _ in range(N) ]
def safe(r, c):
    for i in range(c):
        if board[r][i] == "Q":
            return False
    i, j = r, c
    while i >= 0 and j >= 0:
        if board[i][j] == "Q":
            return False
        i -= 1
        j -= 1
    i, j = r, c
    while i < N and j >= 0:
        if board[i][j] == "Q":
            return False
        i += 1
        j -= 1
    return True
def solve(c):
    if c == N:
        return True
    for r in range(N):
        if safe(r, c):
            board[r][c] = "Q"
            if solve(c + 1):
                return True
            board[r][c] = "."
    return False
```

Question 4: Cab Booking Agent

```
class CabBookingAgent:
    def __init__(self, cab_types):
        self.cab_types = cab_types
    def perceive(self):
        return self.cab_types
    def act(self, choice):
        if choice in self.cab_types:
            return f"Booking Confirmed: {choice}"
        else:
            return "Cab Not Available"
```

Question 5: Uniform Cost Search (UCS) Agent

```
import heapq
class UCSAgent:
    def __init__(self, graph):
        self.graph = graph
    def search(self, start, goal):
        pq = [(0, start, [start])]
        while pq:
            cost, node, path = heapq.heappop(pq)
            if node == goal:
                return cost, path
            for neighbor, c in self.graph[node]:
                heapq.heappush(pq, (cost + c, neighbor, path + [neighbor]))
        return None, None
graph = {
    'S': [('A', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': []
}
```